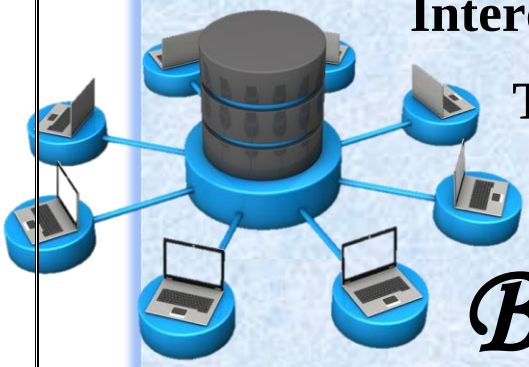
Instituto Politécnico Nacional

UPIITA (Unidad Profesional Interdisciplinaria en Ingeniería y Tecnologías Avanzadas)



Base de Datos Distribuidos

Profesor: De la cruz Sosa Carlos

Asignación: Tarea investigación

Alumno: -Gutiérrez Flores José Julián

-Muñoz Reyes Citlali Ulises

Fecha: 20/03/26

Tema 1: Arquitecturas para bases de datos distribuidas

Tiene 3 arquitecturas de referencia para DBMS (Database Management System), que son client/server system, peer-to-peer distributed DBMS y multidatabase system. Aunque estas arquitecturas cambian a través de los años y varían dependiendo de las necesidades del sistema y de lo que se requiera.

-Sistemas Cliente/Servidor

Los DBMS cliente/servidor se implementaron a principios de la década de 1990 y han tenido un impacto significativo en la tecnología de DBMS. La idea general es muy simple: distinguir la funcionalidad que debe proporcionarse y divide las funciones en dos roles: funciones del servidor (el que provee) y funciones del cliente (el que solicita). Esto proporciona una arquitectura de dos niveles que facilita la gestión de la complejidad de los DBMS modernos y la complejidad de la distribución.

El cliente maneja la interfaz de usuario y la lógica de presentación. Cuando necesita datos, envía una solicitud (usualmente vía SQL) a través de la red. El servidor recibe la petición, procesa la consulta en su motor de base de datos local y devuelve solo el conjunto de resultados. Existen 2 tipos principales de esta arquitectura:

2-Tier (2 capas): El cliente se conecta directamente al servidor.

3-Tier (3 capas): Se introduce un servidor de aplicaciones intermedio que gestiona la lógica de negocio, haciendo que la base de datos sea más segura y escalable.

-Peer-to-Peer

La arquitectura Peer-to-Peer (P2P) es el nivel más alto de descentralización. A diferencia del modelo Cliente/Servidor, donde hay una jerarquía clara, aquí todos los nodos son "iguales" en términos de funcionalidad, es decir, los nodos (ordenadores) actúan simultáneamente como clientes y servidores, compartiendo recursos directamente sin un servidor central

Autonomía Local: Cada nodo gestiona su propio almacenamiento físico y su motor de base de datos local.

Cooperación Global: Aunque son autónomos, los nodos están programados para colaborar. Si un usuario en el Nodo A solicita datos que están físicamente en el Nodo C, el Nodo A sabe cómo pedir esa información de manera transparente.

- Multidatabase System (Sistema de Multibase de Datos)

Un Sistema de Bases de Datos Múltiples (MDBS) se define fundamentalmente por la autonomía total de sus nodos, los cuales operan de manera independiente y a menudo ignoran la existencia de los demás, diferenciándose de los DBMS distribuidos tradicionales en que su integración sigue un enfoque ascendente (bottom-up). En esta arquitectura, el Esquema Conceptual Global (GCS) o "esquema mediado" no es la unión de todas las bases de datos locales, sino una colección selectiva de subconjuntos de datos que cada DBMS local decide compartir mediante un esquema de exportación, permitiendo que los usuarios locales mantengan sus aplicaciones intactas sin verse obligados a participar en la red global. Para gestionar la heterogeneidad entre estos sistemas autónomos, el MDBS suele emplear un modelo de datos canónico que traduce los esquemas externos o conceptuales locales hacia la vista global, garantizando la interoperabilidad sin sacrificar la soberanía de cada base de datos individual.

El Teorema CAP

Para concluir, la elección entre estas arquitecturas está condicionada por el Teorema CAP (Consistency, Availability, Partition Tolerance), el cual establece que un sistema distribuido solo puede garantizar plenamente dos de estas tres propiedades simultáneamente. Mientras que los sistemas Cliente/Servidor tradicionales suelen priorizar la Consistencia (C) y la Disponibilidad (A) en entornos locales, las arquitecturas Peer-to-Peer y los MDBS modernos sacrifican a menudo la consistencia inmediata en favor de la Tolerancia al Particionado (P). Esto significa que el sistema puede seguir operando incluso si hay fallos en la comunicación entre nodos, aceptando una "consistencia eventual" para asegurar que los datos estén siempre disponibles para el usuario final, independientemente de la escala o la ubicación geográfica de la red.

Tema 2: Gestores de bases de datos distribuidas

Los Sistemas de Gestión de Bases de Datos Distribuidas (SGBDD), son un conjunto de múltiples bases de datos lógicamente relacionadas que se encuentran distribuidas en diferentes espacios geográficos o nodos de una red, es decir se presenta como un conjunto de programas empleados para administrar y gestionar la información presente en una base de datos; por esto mismo permiten a los usuarios de negocio almacenar la información, la creación, administración, modificar datos y acceder a los activos de conocimiento de la organización. Asimismo, el gestor de base de datos también se ocupa de realizar consultas y hacer análisis para generar informes.

Estos sirven de interfaz entre la base de datos, las aplicaciones y el usuario, permitiendo recuperar la información de forma sencilla, rápida y eficiente, además, se usan para efectuar análisis y generar informes, de la forma más eficiente posible.

Para funcionar correctamente, los gestores de bases de datos están compuestos por un lenguaje de definición de datos, uno de manipulación y otro de consulta.

Además, los sistemas de gestión de base de datos pueden entenderse como una colección de datos interrelacionados, estructurados y organizados en el ecosistema formado por dicho conjunto de programas que acceden a ellos y facilitan su gestión.

La fragmentación y la replicación son estrategias fundamentales de diseño en estos sistemas para mejorar el rendimiento, la disponibilidad y la escalabilidad.

Replicación de Datos

- Disponibilidad: si uno de los sitios que contiene a la relación r falla, entonces se la puede buscar en otro sitio.
- Incremento del paralelismo: en casos de mayoría de accesos de lectura, los sitios procesando consultas sobre r se ejecutan en simultáneo.
- Incremento del overhead en actualizaciones: el sistema debe asegurar que TODAS las réplicas de la relación r sean consistentes. Cada actualización sobre la relación r debe ser propagada a los sitios conteniendo las respectivas réplicas.

Fragmentación de los datos

- Consiste en dividir una relación r en fragmentos r1, r2,…, rn.
- Estos fragmentos contienen información suficiente como para reconstruir la relación original r.
    - Tipos de fragmentación:

- Fragmentación Horizontal (por filas): separa a una relación r separando las tuplas en subconjuntos.
- Fragmentación Vertical (por columnas): separa a una relación r a través de descomposiciones.
- Combinada: separa a una relación r horizontal y verticalmente.

Gestores de bases de datos relacionales

Se destinan al manejo de bases de datos relacionales, las cuales permiten que cualquier tabla establezca vínculos o relaciones con otra. En este modelo de base de datos la información almacenada se representa en forma de columnas (atributos) y filas (registros). Los atributos especifican el tipo de datos que se manejan y cada registro contiene el valor de este tipo de datos.

El medio de acceso más común a este tipo de gestores de bases de datos es SQL (Structured Query Language) o lenguaje de consulta estructurada.

Las funciones básicas de gestores de bases de datos relacionales son:

- Creación
- Lectura
- Actualización
- Eliminación

Estas permiten crear un sistema bien organizado que impulsa el tratamiento adecuado de los datos.

Algunos de los gestores de bases de datos relacionales más habituales son:

- Oracle MySQL

Este tipo de gestor de datos está basado en código abierto. Destaca por su eficacia, fiabilidad y sencillez, por lo que se emplea en muchas páginas web en la actualidad.

- Microsoft SQL Server

Se basa en el lenguaje Transact-SQL e incorpora un conjunto de extensiones de programación propias de lenguaje estándar.

- MariaDB

Este SGBD es una bifurcación de MySQL, por lo que ofrece la mayoría de los beneficios de este último. Es un gestor de código abierto, por lo que se puede emplear de manera gratuita.

Gestores de bases de datos no relacionales

Esta clase de sistemas no requiere emplear estructuras de datos fijos, como las tablas. Por este motivo, son muy útiles para manejar grandes cantidades de información manteniendo una buena velocidad de respuesta. Se caracterizan

porque no son rígidas, permiten gestionar la información con una alta escalabilidad horizontal y emplean muchos más nodos que los gestores de bases de datos relacionales. Por todo ello, se utilizan en aplicaciones que necesitan leer y/o escribir grandes volúmenes de datos.
Entre los principales gestores de bases de datos no relacionales empleados destacan:

- MongoDB

Es uno de los gestores de base de datos no relacionales más populares del momento. Está orientado a ficheros y almacena la información en estructuras BSON; es decir, en un formato de serialización binaria. Utiliza un esquema flexible y de escalabilidad vertical y horizontal, por lo que se pueden agregar otros servidores sin afectar su rendimiento lo que le facilita la agregación de otros servidores sin que se afecte el rendimiento.

- Redis

Basado en el almacenaje clave valor, se utiliza principalmente para guardar datos en memoria caché y administrar sesiones.

- Cassandra

Este sistema está diseñado para administrar cantidades masivas de datos distribuidos en un gran número de máquinas. Emplea almacenamiento clave-valor, es distributivo, masivamente escalable y utiliza un lenguaje de consulta propio llamado CQL (Cassandra Query Language).

- DynamoDB

Es un gestor de base de datos del servicio AWS (Amazon Web Services), el cual acepta modelos de datos de documentos y clave-valor. Debido a su configuración, ofrece tiempos de respuesta bajos y una gran escalabilidad.

Gestor	Tipo de Datos	Soporte de Distribución	Costo / Acceso
Apache Cassandra	**No Relacional** (Columnar)	**Nativo.** Usa un anillo de nodos donde la fragmentación y replicación son automáticas.	**Acceso Libre** (Open Source)
MongoDB	**No Relacional** (Documental)	**Nativo.** Utiliza "Sharding" para fragmentar datos y "Replica Sets" para copias.	**Libre** (Community) / **Pago** (Atlas/Enterprise)
PostgreSQL	**Relacional** (SQL)	**Extensiones.** Soporta replicación nativa. Para fragmentación usa extensiones como *Citus*.	**Acceso Libre** (Open Source)
MySQL	**Relacional** (SQL)	**Configurable.** Soporta replicación Maestro-Esclavo. Para fragmentación usa *MySQL Cluster*.	**Acceso Libre** (GPL) / **Pago** (Enterprise)
Google Spanner	**Relacional** (SQL)	**Total.** Diseñado desde cero para ser distribuido globalmente con consistencia fuerte.	**Pago** (Cloud Service)
Redis	**No Relacional** (Clave-Valor)	**Nativo.** Usa *Redis Cluster* para distribuir datos entre varios nodos.	**Acceso Libre** / **Pago** (Enterprise)
Oracle Database	**Relacional** (SQL)	**Avanzado.** Usa *Oracle Sharding* y *Data Guard* para replicación de alto nivel.	**Pago** (Licencia Comercial)

Al momento de elegir el sistema gestor de base de datos más adecuado debe tomarse en cuenta el volumen de información que se almacenará, el tipo de datos a guardar, el presupuesto disponible y cómo se gestionará la información, entre otros puntos.

Fuentes de consultas:

- Özsu, M. T., & Valduriez, P. (2020). Principles of Distributed Database Systems. Springer. (Teoría base de MDBS y P2P).
- IBM. (s. f.). CAP https://www.ibm.com/mx-es/think/topics/cap-theorem

  (Limitaciones teóricas).

- Reactive Programming. (2018). Estilos Arquitectónicos. https://reactiveprogramming.io/blog/es/estilos-arquitectonicos/p2p

  (Topologías y algoritmos de red).

- *Gestores de base de datos: qué son y qué tipos existen*. (2023, febrero 24). Universidad Internacional de La Rioja. https://mexico.unir.net/noticias/ingenieria/gestores-de-base-de-datos/
- Pérez, S. D. (2021, septiembre 8). Gestor de Base de datos: Qué es, Funcionalidades y Ejemplos. *Intelequia*. https://intelequia.com/es/blog/post/gestor-de-base-de-datos-qu%C3%A9-es-funcionalidades-y-ejemplos
- *Los sistemas gestores de BDD*. (s/f). Google.com. Recuperado el 20 de marzo de 2026, de https://sites.google.com/view/basededatosdistribuidasaldeni/conceptos-de-base-de-datos-distribuidas/los-sistemas-gestores-de-bdd
- De una, U. B. de D. D. C. (s/f). *Bases de Datos Distribuidas*. Edu.ar. Recuperado el 20 de marzo de 2026, de https://cs.uns.edu.ar/~gis/ebd/Archivos/Clases/EBD%20-%20Clase%2021%202004%20BN.pdf
- (S/f). Edu.mx. Recuperado el 20 de marzo de 2026, de https://cursos.clavijero.edu.mx/cursos/146_fbd/modulo1/contenidos/documentos/fundamentos-de-sistemas-de-bases-de-datos.pdf